■ StaffController.java Explanation (Tanglish) - Colored Version

■ Package Declaration

package com.inventory.controller;
➡■ Ithu controller package la iruka class, staff related APIs handle panna use agum.

■ Imports

➡■ Staff model & StaffService import panniruka.
➡■ Spring Boot annotations (RestController, RequestMapping, etc.) use panniruka.
➡■ Lombok @RequiredArgsConstructor use panniruka – automatic constructor create aagum final fields ku.

■■ Class Declaration

@RestController – REST API controller nu indicate pannum.
@RequestMapping('/api/staff') – API base URL set pannum.
@RequiredArgsConstructor – automatic constructor generation.
@CrossOrigin('*') – allow all frontend origins to access this API.

■ Dependency Injection

private final StaffService staffService;
➡■ Spring automatically constructor injection pannum (@RequiredArgsConstructor thanks).

■ 1. Get All Staff

@GetMapping
➡■ Database la iruka ella staff list fetch pannum using staffService.getAllStaff().

■ 2. Get Staff by ID

@GetMapping('/{id}')
➡■ PathVariable use panni specific ID staff fetch pannum.
➡■ ResponseEntity.ok() la wrap panniruka for clean response.

■ 3. Create New Staff

@PostMapping
➡■ @RequestBody la frontend send panna staff details receive pannum.
➡■ staffService.createStaff(staff) call pannum.
➡■ ResponseEntity.status(HttpStatus.CREATED) return pannum (201).

■ 4. Update Existing Staff

@PutMapping('/{id}')
➡■ Existing staff details update panna use pannum.
➡■ staffService.updateStaff(id, staff) call pannum.

■ 5. Delete Staff

@DeleteMapping('/{id}')
➡■ staffService.deleteStaff(id) call pannum.
➡■ Return ResponseEntity.noContent() – means 204 No Content (successful deletion).

■ Summary Flow

1■■ Get all staff – `/api/staff` (GET)
2■■ Get staff by ID – `/api/staff/{id}` (GET)
3■■ Add new staff – `/api/staff` (POST)
4■■ Update staff – `/api/staff/{id}` (PUT)
5■■ Delete staff – `/api/staff/{id}` (DELETE)

■ Simple, clean CRUD controller for staff management.